Special Module: Methods in Mol Ecol and Pop Gen

Rebecca S. Chen

Table of contents

1 Methods in Mol Ecol and Pop Gen **3**

2 Introduction **4**

3 Basic skills - a test **6**
3.1 Datacamp 6

4 Week 7 inbreeding **16**
4.1 Background 16
4.2 The dataset 16
4.3 Microsatellite datasets 17
4.4 Calculating sMLH 17
4.5 Exercise 1 (day 1) 17
4.6 Exercise 2 (day 1) 19
4.7 Exercise 3 (day 2) 25

5 Week 9 NGS **27**

6 Day 1: sequencing methods, fasta files and reference genomes **28**
6.1 Overview 28
6.2 Exercise 1: sequencing 28
6.3 Exercise 2: read data 29
6.4 Exercise 3: explore fq.gz file 30
6.5 Exercise 4: reference genomes 35
6.6 Exercise 5: alignments 36

7 Day 2: single nucleotide polymorphisms **37**
7.1 Overview 37
7.2 Exercise 1: loading in .vcf files 37
7.3 Exercise 2: cleaning the file 39
7.4 Exercise 3: handling and filtering 39
7.5 Exercise 4: comparing SNPs and microsatellites 41

8 Week 10 ROHs **43**
8.1 Analysis 2 - SNPs 43

1 Methods in Mol Ecol and Pop Gen

2 Introduction

This webpage/document contains all the practical materials for the Spezial Modul entitled "Methods in Molecular Ecology and Population Genetics".

The order of the practicals/tests on this webpage are in the order of the way we will discuss them throughout the course. Please don't hestitate to ask David or Rebecca any questions throughout the course.

If anything is unclear or not working, please let us know!

Each session will contain text, exercises and answers. We encourage you to really think before you reveal the answer, otherwise you won't learn. If you need any help getting to the answer, you can also ask us questions to put you in the right direction. The same applies to helping you troubleshoot, of course.

Remember, even the experts will google how to code things. Once you get more fluent in the language, you can do simple things by heart. But just like learning any langauge, this takes time and practise. So don't be afraid to look into your notes or look up how to do things, this is how every person works in R

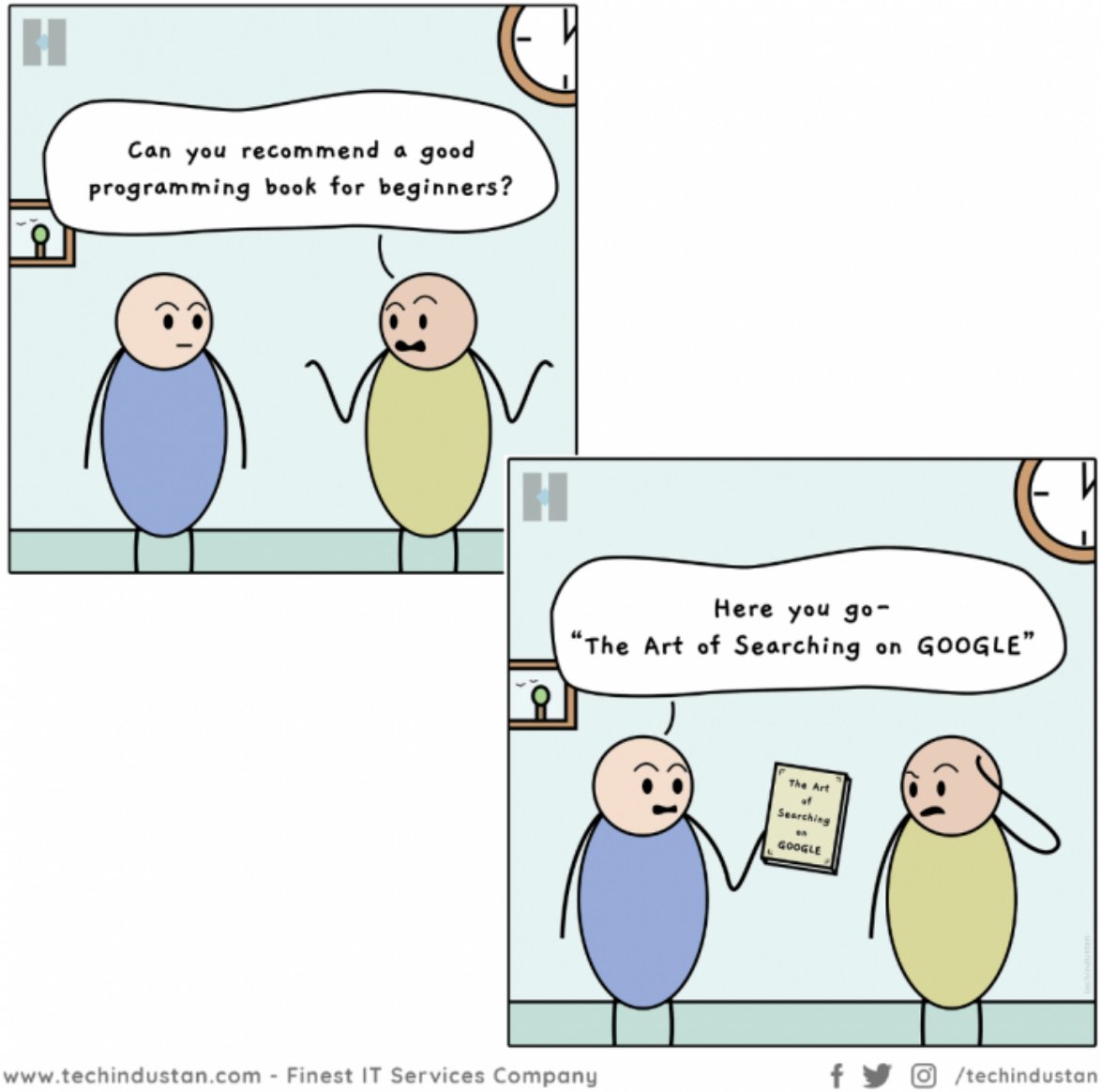


Figure 2.1: Use the resources available to you!

3 Basic skills - a test

3.1 Datacamp

In the first week, you will have almost two full days to learn the basics of R yourself. We have given you access to DataCamp, a great interactive platform to learn R at any level. The courses we would like you to complete because they prepare you with all you need to know to follow the rest of the course are as follows:

- "Introduction to R"
- "Introduction to tidyverse"
- "Introduction to dplyr"

Please follow them at your own speed. If you think something is too easy because you already have some prior experience, you can skip through it and learn at your own speed. Feel free to start other courses too. All we want you to be able to do, is gain the skills and experience to be able to complete the exercises below. These exercises are therefore not graded in any way, it's just a way to make sure you understand the basics to continue with the rest of the course.

! A note on AI

You can find the answers and code when you click on the toggle, but try to do as much as you can without looking at the answers first. You can also see that you will find the answer and then the code, after clicking on it. This is another way in which you can get a 'hint' - i.e. the answer, to make you think about what kind of code you need to receive such an answer.
You can use Google or your notes from DataCamp to find the answers, but please don't use ChatGPT or other AI tools to help you find the answers directly. For example, don't just put the question as a prompt, but you could ask: how do I calculate the mean in R? Or: how can I make a basic plot with ggplot2 to visualise XYZ? In other words: don't 'cheat', but if you want to use ChatGPT, use it in a way that will help you learn and it doesn't hand the answer to you.

3.1.1 Data description

Let's work with some global height dataset. It was downloaded from the website https://ncdrisc.org/data-downloads-height.html and the file can be found in the `data` directory under `week_4_test`. The data contain the mean heights of adolescents of both boys and girls at different age groups. The mean height is the mean height of a certain age group in a specific year.

3.1.2 Part 1: exploration

- The dataset is called `NCD_RisC_Lancet_2020_heigh_child_adolescent_global.csv` and is stored in the `data` directory under `week_4_test`. What does the `.csv` extension mean?

Answer

Csv stands for comma separated file. It is a file format to store tables (such as Excel files), where each line represents a data record, and fields are separated by commas. CSV files are commonly used to store data, but you have to be careful if your data contains text that might include comma's, because it will think the sentence needs to be split up into different fields! Other ways to separate fields are for example by using tabs (`) or spaces.

- Import the data set and assign it to a variable (called e.g. `data`)

Code and answer

We are working within an R project, so we can use relative path names. If you prefer using the absolute path name, this might look different. Also, path specification is dependent on your operating system. This has been done in iOS (Apple), but Windows looks different.

```
# using read.csv
data <-
 ↪  read.csv("../data/week_4_test/NCD_RisC_Lancet_2020_height_child_adolescent_global.csv")

# or absolute path name
data <-
 ↪  read.csv("/Users/vistor/Documents/Work/GitHub/postdoc/special_module_molecol_popgen/dat
 ↪
```

```r
# or using read table, make sure you specify there's a header!
data <-
 ↪  read.table("../data/week_4_test/NCD_RisC_Lancet_2020_height_child_adolescent_global.csv
 ↪  sep = ",", header=T)
```

- How many data entries are there?

Number of data entries = number of rows, because we are dealing with a data table.

Code and answer

```r
nrow(data)
```

```
[1] 1050
```

So 1050 entries = this excludes the header.

- What is the youngest age group in the dataset, and what is the oldest age group (i.e. how old are those groups)?

Code and answer

First we need to know what columns there are in the data and what they are called, which can be done in different ways.
Remember you can always view the data (by clicking on it in the environment or by calling `View(data)`).

```r
# this shows you just the structure of the table
str(data)
```

```
'data.frame':   1050 obs. of  5 variables:
 $ Sex                       : chr  "Boys" "Boys" "Boys" "Boys" ...
 $ Year                      : int  1985 1985 1985 1985 1985 1985 1985 1985 1985 1985 ...
 $ Age.group                 : int  5 6 7 8 9 10 11 12 13 14 ...
 $ Mean.height               : chr  "105.6490519" "111.6353523" "117.1894908" "122.4381031"
 $ Mean.height.standard.error: chr  "0.258115469" "0.227526225" "0.210784409" "0.199455168"
```

```r
# this tells you the column names of the table
names(data)
```

```
[1] "Sex"                        "Year"
[3] "Age.group"                  "Mean.height"
[5] "Mean.height.standard.error"
```

```r
# this shows you the first 6 entries and the column names
head(data)
```

```
   Sex Year Age.group Mean.height Mean.height.standard.error
1 Boys 1985         5 105.6490519                0.258115469
2 Boys 1985         6 111.6353523                0.227526225
3 Boys 1985         7 117.1894908                0.210784409
4 Boys 1985         8 122.4381031                0.199455168
5 Boys 1985         9 127.4997942                0.194521726
6 Boys 1985        10 132.7080987                0.187002286
```

From this we can see that the column that stores the age is called `Age.group`

```r
# minimum age i.e. youngest age class
min(data$Age.group)
```

```
[1] 5
```

```r
# maximum age i.e. oldest age class
max(data$Age.group)
```

```
[1] 19
```

So 5 and 19, respectively.

3.1.3 Part 2: cleaning the data

- What is the data type / class of the mean height and its standard error?

Code and answer

```r
# this tells us the class of a specific column
class(data$Mean.height)
```

```
[1] "character"
```

```r
class(data$Mean.height.standard.error)
```

```
[1] "character"
```

```r
# this tells us the class of all columns
str(data)
```

```
'data.frame':   1050 obs. of  5 variables:
 $ Sex                       : chr  "Boys" "Boys" "Boys" "Boys" ...
 $ Year                      : int  1985 1985 1985 1985 1985 1985 1985 1985 1985 1985 ...
 $ Age.group                 : int  5 6 7 8 9 10 11 12 13 14 ...
 $ Mean.height               : chr  "105.6490519" "111.6353523" "117.1894908" "122.4381031"
 $ Mean.height.standard.error: chr  "0.258115469" "0.227526225" "0.210784409" "0.199455168"
```

Both are characters/strings.

- Hmm, this data class is probably incorrect… It should be numeric, right? What's causing this?

Code and answer

A classic coding issue, "what's up with my data"? Figuring out why something is not behaving as expected will happen most of the time you are coding. How do we diagnose the problem?
We can start by having a look at the data using e.g. `View(data)` and by doing some tests.

```
# let's see what happens when we change the data class to numeric, and
↪  store this in a test variable.

test <- as.numeric(data$Mean.height)

Warning: NAs introduced by coercion
```

It gives a warning which indicates it could not change one or more of the values to numeric so it had to be put to NA. Why?
What sometimes works is to sort the column, this might help us diagnose what is happening in such a large table. When we look at `View(data)` / the data tab, we can click on the column `Mean.height` and it will sort it. Click on it again, and it will sort it in reverse order
Whatever way you used, you might have identified the issue!
There are two "ERRORS" at row number 112, a boy of age class 1992 and age 11. This text is causing the error, as R doesn't recognise this column as numeric now.

- Fix the error and change the data class to numeric

Code and answer

We can't correct the value because we don't know what happened with this boy's height measurement. So let's either remove the entire row or change the values to NA. You could do this manually by going into the raw data using e.g. Excel. But a better practise is by doing this in the code - this way we can always track exactly what we did, and others can see it too because it is documented.
In the first method below, we simply remove the 112'th row, so we remove the entire data point. It's always smart to store something in a different variable if you remove something so you do not override the original data, I'm showing you an example here but we'll continue with the 'data' variable as we know it's all good.
In the second and third method, we remove the text "ERROR" and instead replace it with the 'official' NA value that R recognises as NA.

```
## method 1
# remove the row
data_clean <- data[-112,]

# then we change the data class to numeric and we do not get a warning
data_clean$Mean.height <- as.numeric(data_clean$Mean.height)
# same for the SE
data_clean$Mean.height.standard.error <-
 ↪  as.numeric(data_clean$Mean.height.standard.error)

## method 2
# this is what we had done before and automatically put the 'character'
 ↪  ERROR as NA, you'll get a warning but now we know why

data$Mean.height <- as.numeric(data$Mean.height)

Warning: NAs introduced by coercion

# same for the SE
data$Mean.height.standard.error <-
 ↪  as.numeric(data$Mean.height.standard.error)

Warning: NAs introduced by coercion

## method 3

# this is the same as above, but manually which is a bit longer and more
 ↪  complex
```

```r
data$Mean.height[which(data$Mean.height=="ERROR")] <- NA
data$Mean.height.standard.error[which(data$Mean.height.standard.error=="ERROR")]
↪  <- NA

# change to numeric, now we won't get a warning
data$Mean.height <- as.numeric(data$Mean.height)
data$Mean.height.standard.error <-
↪  as.numeric(data$Mean.height.standard.error)
```

There we go! It's always good to check if things worked, so you can use e.g. `str(data)` now to check that R indeed changed the columns to numeric.

3.1.4 Part 3: summary statistics

- What is the mean across years of the mean heights in the dataset?

Code and answer

The function that calculates means is, intuitively, `mean()` which is a base function (i.e. you don't need to install a package for this). If you want to know more details about how to use a function, you can use `?mean`

```r
# we have to make sure we remove the NA when calculating the mean though!
mean(data$Mean.height, na.rm = T)
```

```
[1] 141.7635
```

```r
# or we use the data that doesn't contain the NA
mean(data_clean$Mean.height)
```

```
[1] 141.7635
```

- What is the mean across years of the mean height, calculated separately for boys and girls?

Code and answer

Here's where your preference can kick in: do you want to use dplyr or something else, like base R or data.table? I will show you multiple options, just see what looks more intuitive to you. To make sure we can use dplyr, we have to load the library.
To answer this question, we have to do multiple steps: divide the data among the sexes

somehow, and calculate the mean. This can be done in steps, or in one go. If you're a computational scientists, you would want things to be done in as little code as possible. But since we are learning, just do what makes most sense to you.

```r
# method 1 with dplyr
library(dplyr) # we can use dplyr to use the filter option
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
girls <- data %>% filter(Sex == "Girls")
boys <- data %>% filter(Sex == "Boys")

mean(girls$Mean.height, na.rm = T)
```

```
[1] 140.02
```

```r
mean(boys$Mean.height, na.rm = T)
```

```
[1] 143.5103
```

```r
# method 2 without dplyr
mean(data$Mean.height[which(data$Sex == "Girls")], na.rm=T)
```

```
[1] 140.02
```

```r
mean(data$Mean.height[which(data$Sex == "Boys")], na.rm=T)
```

```
[1] 143.5103
```

- In what year was the standard error lowest for boys and girls separately, for the age class 12? (i.e. the variance smallest?)

Again, we need to take multiple steps here: only look at data from the Age group 12, and then look at what year the variance was the smallest (i.e. the minimal variance).

I'll only show you the dplyr way, but you can use many different methods to get to the same answer here.

Code and answer

```
# method 1: with dplyr
girls_12yr <- girls %>% filter(Age.group == 12)
boys_12yr <- boys %>% filter(Age.group == 12)

girls_12yr %>% arrange(Mean.height.standard.error) %>% head() # 2003,
 ↪  with a SE of 0.09367336

    Sex Year Age.group Mean.height Mean.height.standard.error
1 Girls 2003        12    146.6524                 0.09367336
2 Girls 2004        12    146.7459                 0.09369505
3 Girls 2005        12    146.8387                 0.09382388
4 Girls 2002        12    146.5615                 0.09397351
5 Girls 2006        12    146.9259                 0.09423025
6 Girls 2001        12    146.4734                 0.09451832

boys_12yr %>% arrange(Mean.height.standard.error) %>% head() # 2004, with
 ↪  a SE of 0.1342430

   Sex Year Age.group Mean.height Mean.height.standard.error
1 Boys 2004        12    144.8178                 0.1342430
2 Boys 2003        12    144.7568                 0.1343755
3 Boys 2005        12    144.8627                 0.1347293
4 Boys 2002        12    144.6810                 0.1349744
5 Boys 2006        12    144.8881                 0.1355101
6 Boys 2001        12    144.5841                 0.1360441
```

3.1.5 Part 4: plotting

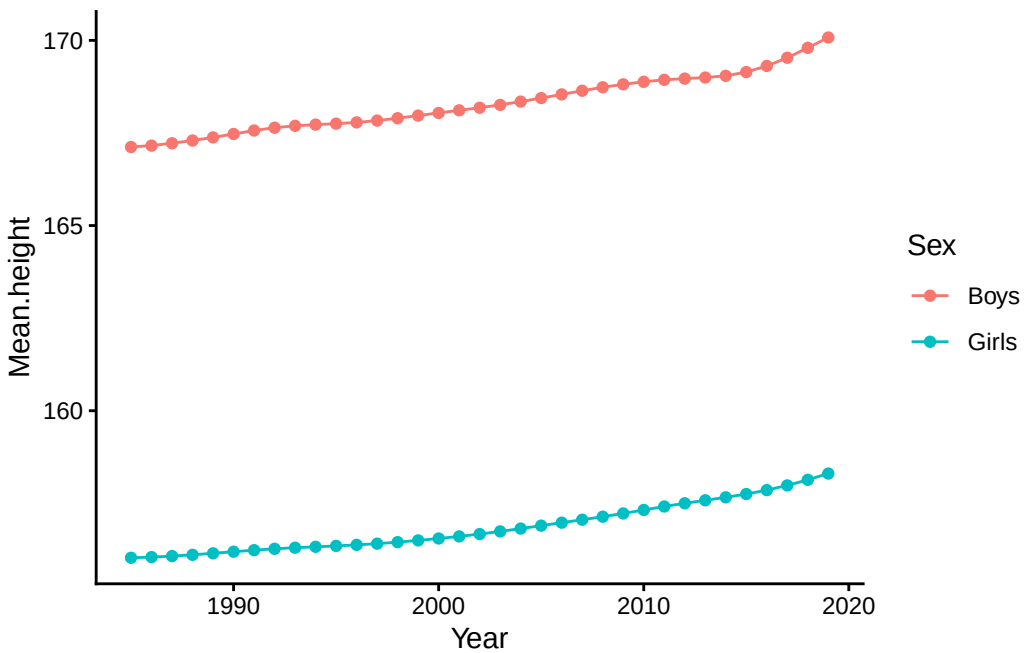
- Make a plot that shows you whether the mean height of 18 year olds increased or decreased over time. Bonus if you do it seperately per sex!

Code and answer

Let's get plotting! First choose: do you want to use ggplot2 (recommended) or base R to make your plot?
Before you start making code, think what elements should be on the axes. What should be on the X axis, what variable on the Y axis, and what type of graph would represent

the data best? A boxplot, scatterplot, line graph, pie chart? Choose the right element accordingly and then start writing the code.
We also have to use a portion of the data only, again. We can use dplyr (`filter()`) but I will also show you another function here, `subset()` which is a base R function to do something similar to `filter()`.

```
# method 1: with ggplot
library(ggplot2)

ggplot(subset(data, Age.group == 18), aes(x = Year, y = Mean.height)) +
 ↪  geom_point(aes(col = Sex)) + geom_line(aes(col = Sex)) +
 ↪  theme_classic()
```




```
# method 2: base R -- not sure how to do it
```

As we see, both the mean height of girls and boys aged 18 years increased over time. What is also clear is that base R is a lot longer, and you require a lot of separate elements.

Well done! If you managed to answer all of the above, with or without some help from your notes and the internet, you're ready to explore genetic data and answer some pop gen questions.

4 Week 7 inbreeding

We've just learned what inbreeding is and how we can measure it. In this practical, we will learn how to quantify heterozygosity (the inverse of inbreeding) using a real-life dataset.

In the analysis, we will use a microsatellite dataset to quantify the inbreeding coefficient sMLH using the package `inbreedR`. In the second analysis, we will identify homozygous stretches in the genome (HBD segments - homozygous by descent / ROHs - runs of homozygosity) from SNP data using the package `RZooRoh`.

4.1 Background

The dataset we are working with are microsatellite genotypes from black grouse (*Lyrurus tetrix*), a lekking bird. The dataset was taken from this paper which investigated the effects of hunting on inbreeding and gene flow. It is a large microsatellite dataset from female and male birds as well as chicks, sampled at 12 populations in Central Finland. For some populations, recreational hunting is allowed whereas others are protected.

Inbreeding in this species is generally avoided by female-biased dispersal and non-overlapping generations (i.e. females have a shorter lifespan so will have probably died by the time she could hypothetically mate with her offspring). However, due to habitat fragmentation and destruction, inbreeding can still occur. The black grouse is not a threatened species and lives across all over Europe (including Germany). Nevertheless, it is locally threatened: in Germany there is only one population left: in the Lüneburger Heide. It is therefore important to understand inbreeding dynamics in this species.

4.2 The dataset

Blood samples were collected between 2001 and 2007 from a total of 1878 adult black grouse, of which 1065 were males and 813 were females. The birds were captured in walk-in traps baited with oats, aged as yearlings or older according to the shape of the outmost primaries and marked with metal and colour rings for future identification. Blood (1–2 ml) was taken with a heparinized syringe from the brachial vein. After centrifugation, the red blood cells were stored in 70% ethanol at 4°C for subsequent DNA analysis.

In addition to the adults, 1370 chicks were sampled (370 females, 325 males and 675 individuals of unknown sex) from 200 different broods. The chicks were sampled at seven of the 12 sites between 2001 and 2006. Nesting sites were located after the lekking season and the hatching date of the chicks was estimated by floating the eggs in warm water. The chicks were then captured on the approximate day of hatching (in May/June) for blood sample collection where possible, and if not, egg shells were collected instead.

4.3 Microsatellite datasets

Microsatellite data can be stored in various ways. The most common way to note them down is: one row per individual, the loci (and its two alleles) in columns. Which means that for each locus, you have two columns: one for each allele, which is commonly noted as allele a and allele b. The value stored in these cells is the fragment length in base pairs, which therefore reflects the number of times the specific motif (e.g. AT) is repeated in a given individual at a given locus.

4.4 Calculating sMLH

Using this dataset, we will calculate the standardized multilocus heterozygosity. Standardized Multilocus Heterozygosity (sMLH) is a measure of an individual's genetic diversity that divides their total number of heterozygous loci by the average observed heterozygosity of those specific, successfully typed loci in the population. What we are essentially looking at is whether two alleles are the same in length and thus homozygous, or whether the two alleles are different in length and thus heterozygous. sMLH measures heterozygosity: this is therefore the opposite of homozygosity. In other words, individuals with lower sMLH values are more inbred and vice versa. The paper that describes how to calculate sMLH with the `inbreedR` package can be found here.

4.5 Exercise 1 (day 1)

- Load in the dataset. It can be found under `data/week_5_inbreeding` and is named msat_black_grouse - it is a .csv file with a header.
- Investigate the structure of the dataset: how many microsatellite loci does the dataset contain? And how many individuals are there in the dataset? Also make sure to check whether all 12 populations are included in this dataset

Code and answer

```r
# load in the dataset
data <- read.csv("../data/week_5_inbreeding/msat_black_grouse.csv")
head(data)
```

```
    id pop BG16a BG16b BG18a BG18b BG15a BG15b BG19a BG19b BG6a BG6b TTT1a
1 D005  12   156   164   148   148   189   189   169   177  199  211   230
2 D009   4   156   164   152   156   185   189   165   177  283  283   238
3 D023   2   156   168   156   160   185   189   173   185  199  207   230
4 D035  12   156   160   156   160   181   193   173   181  203  203   226
5 D038  11   156   168   144   164   181   189   169   169  203  203   242
6 D057   4   164   164   152   164   189   189   169   177  203  207   242
  TTT1b TTD2a TTD2b TTD3a TTD3b TUD6a TUD6b TUT3a TUT3b TUT4a TUT4b TTT2a TTT2b
1   246   149   149   136   138   184   186   156   160   173   173    NA    NA
2   242   149   149   128   152   180   182   172   176   165   169    NA    NA
3   250   149   153   152   152   186   194   168   168   161   169    NA    NA
4   242   149   149   146   156   180   186   160   168   157   165    NA    NA
5   242   149   149   138   154   186   186   160   168   169   181    NA    NA
6   250   149   153   128   146   180   192   156   156   157   161    NA    NA
```

```r
# number of loci
(ncol(data) - 2)/2 # there are two other columns 'id' and 'pop' which are
 ↪  not loci, and each locus has two alleles
```

```
[1] 12
```

```r
# number of individuals
nrow(data)
```

```
[1] 2078
```

```r
# checking the number of populations
length(unique(data$pop))
```

```
[1] 12
```

```r
# or
summary(as.factor(data$pop))
```

```
  1   2   3   4   5   6   7   8   9  10  11  12 
274 364  46 210 320  59  62  40  91 184 350  78 
```

Number of loci: 12 Number of individuals: 2078 Yes, data from all 12 populations are included

4.6 Exercise 2 (day 1)

Now, we want to calculate sMLH based on this dataset. We will do this using the R package `inbreedR`. Make sure to familiarise yourself with the package and read through the manual / paper.

After reading through it, execute the following:

- There are a loci that have NAs. Which one is it / are they? Do you think it's a problem when we want to calculate sMLH?

Code and answer

```
#easiest way to look across your dataset is with summary()
summary(data) # at the bottom you will see the number of NA rows in each
 ↪  column

      id                 pop             BG16a           BG16b
 Length:2078        Min.   : 1.000   Min.   :  0.0   Min.   :  0.0
 Class :character   1st Qu.: 2.000   1st Qu.:152.0   1st Qu.:160.0
 Mode  :character   Median : 5.000   Median :156.0   Median :164.0
                    Mean   : 5.839   Mean   :155.6   Mean   :161.9
                    3rd Qu.:10.000   3rd Qu.:160.0   3rd Qu.:164.0
                    Max.   :12.000   Max.   :172.0   Max.   :198.0
                                     NA's   :5       NA's   :5
    BG18a           BG18b           BG15a           BG15b
 Min.   :  0.0   Min.   :  0.0   Min.   :169.0   Min.   :181.0
 1st Qu.:152.0   1st Qu.:160.0   1st Qu.:185.0   1st Qu.:189.0
 Median :156.0   Median :164.0   Median :185.0   Median :189.0
 Mean   :155.8   Mean   :163.2   Mean   :186.1   Mean   :191.2
 3rd Qu.:160.0   3rd Qu.:168.0   3rd Qu.:189.0   3rd Qu.:193.0
 Max.   :172.0   Max.   :192.0   Max.   :197.0   Max.   :221.0
 NA's   :3       NA's   :3
    BG19a           BG19b            BG6a            BG6b         TTT1a
 Min.   :161.0   Min.   :161.0   Min.   :187.0   Min.   :195   Min.   :218.0
 1st Qu.:165.0   1st Qu.:173.0   1st Qu.:199.0   1st Qu.:203   1st Qu.:234.0
 Median :169.0   Median :177.0   Median :199.0   Median :207   Median :238.0
 Mean   :169.5   Mean   :175.7   Mean   :200.6   Mean   :215   Mean   :238.5
 3rd Qu.:173.0   3rd Qu.:177.0   3rd Qu.:203.0   3rd Qu.:211   3rd Qu.:242.0
 Max.   :185.0   Max.   :193.0   Max.   :287.0   Max.   :299   Max.   :258.0
                                                               NA's   :4
    TTT1b           TTD2a           TTD2b           TTD3a
 Min.   :222.0   Min.   :149.0   Min.   :149.0   Min.   :  0.0
```

```
 1st Qu.:242.0   1st Qu.:149.0   1st Qu.:149.0   1st Qu.:134.0
 Median :246.0   Median :149.0   Median :149.0   Median :138.0
 Mean   :245.6   Mean   :149.2   Mean   :150.1   Mean   :138.3
 3rd Qu.:250.0   3rd Qu.:149.0   3rd Qu.:149.0   3rd Qu.:146.0
 Max.   :258.0   Max.   :186.0   Max.   :198.0   Max.   :160.0
 NA's   :4       NA's   :1138    NA's   :1138    NA's   :4
     TTD3b           TUD6a           TUD6b           TUT3a
 Min.   :  0.0   Min.   :  0.0   Min.   :  0.0   Min.   :144.0
 1st Qu.:146.0   1st Qu.:180.0   1st Qu.:184.0   1st Qu.:156.0
 Median :146.0   Median :182.0   Median :186.0   Median :160.0
 Mean   :147.3   Mean   :182.4   Mean   :185.4   Mean   :160.3
 3rd Qu.:152.0   3rd Qu.:184.0   3rd Qu.:186.0   3rd Qu.:164.0
 Max.   :172.0   Max.   :192.0   Max.   :200.0   Max.   :176.0
 NA's   :6       NA's   :2       NA's   :2       NA's   :1
     TUT3b           TUT4a         TUT4b         TTT2a         TTT2b
 Min.   :152.0   Min.   :  0   Min.   :  0   Min.   :149   Min.   :149
 1st Qu.:164.0   1st Qu.:161   1st Qu.:169   1st Qu.:149   1st Qu.:149
 Median :168.0   Median :165   Median :169   Median :149   Median :149
 Mean   :166.6   Mean   :165   Mean   :171   Mean   :149   Mean   :150
 3rd Qu.:168.0   3rd Qu.:169   3rd Qu.:173   3rd Qu.:149   3rd Qu.:149
 Max.   :180.0   Max.   :185   Max.   :189   Max.   :157   Max.   :157
 NA's   :1                                   NA's   :213   NA's   :213
```

We see a few loci with a few NA's (<10): BG16, BG18, TTT1, TTD3, TUD6. But there is one locus with huge number of NA's (>1000) which is TTD2 and another with a significant amount of data missing: TTT2. But this is what's important about the `s` in `sMLH`: it calculates homozygosity across multiple loci but accounts for missing data. So it inheritently only looks at loci that have been successfully genotyped and therefore have a value. It is therefore fine for the calculation.
However, if you sample less loci, you have a lower resolution. It is therefore important to investigate your data like this: we want to make sure that the number of loci used to calculate sMLH does not bias the sMLH values. Say all individuals in this dataset are homozygous for TTD2 (hypothetically), then the sMLH values of those individuals that were not genotyped at TTD2 might all have slighly higher sMLH values. It is good to investigate this later on.
The short answer: no, this missing data does not matter for the calculation of sMLH, but we should keep an eye on it so we make sure this doesn't bias our results.

Before we calculate sMLH, we should clean up some of the data. Execute the following (not necessarily in this order):

- The populations currently have a number: that's not ideal. Replace them with the actual location names which can be found in the .csv file stored in `data/week_5_inbreeding`

titled `metadata_populations`. The second column `pop_num` refers to the number in the microsatellite dataset stored in the `pop` variable.

- csv files don't like special characters: the names of population 1 and 5 therefore look weird. Change these names to Koskenpää and Nyrölä, respectively

Code and answer

```
# load in the names of the populations
pops <- read.csv("../data/week_5_inbreeding/metdata_populations.csv")
head(pops)

              pop pop_num     hunt        X        Y
1 Koskenp\xe4\xe4       1   hunted 62.03983 25.20602
2       Kummunsuo       2 unhunted 62.34530 24.76298
3       Lauttasuo       3   hunted 62.50857 24.87211
4        Lehtusuo       4 unhunted 62.25780 24.96876
5    Nyr\xf6l\xe4       5 unhunted 62.32777 25.46674
6         Palosuo       6   hunted 62.38357 25.27636

# change the names
pops[1,1] <- "Koskenpää"
pops[5,1] <- "Nyrölä"

# approach 1: merge the two files and then only keep the written out
 ↪  population, not the number
## load dplyr
library(dplyr)


Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union

## merge the pops file with the microsatellite data
data_clean <- left_join(data, pops, by = c("pop" = "pop_num"))
## reorganise the columns
data_clean <- data_clean[,c(1,27,3:26)]
```

```r
## rename the column to pop instead of pop_num
names(data_clean)[2] <- "pop"

# approach 2: use a lookup table / using match
## replace numeric pop codes with letters via lookup
data$pop <- pops$pop[match(data$pop, pops$pop_num)]
```

So, now that the data are cleaned up and ready to analyse: let's calculate sMLH. Use the vignette to guide you through this process.

Tips

If you're struggling to find out how to do this, you can use these steps: 1. inbreedR requires a certain format: the ids have to be the rownames, and the data should only contain the microsatellite genotypes (so no population info) 2. to convert our dataframe to an inbreedR data format, you can use the function `convert_raw` from the inbreedR package. This function converts the genotypes to a binary distinction: 1 stands for heterozygous, 0 for homozygous for each of the loci. 3. to calculate sMLH, we need the `sMLH` function

Code and answer

```r
## Change formats to be loaded into inbreedR (binary where rownames are
 ↪  id's and no populations)
library(tibble)

Warning: package 'tibble' was built under R version 4.3.3

data_raw <- data_clean
data_raw <- data_raw %>% remove_rownames %>% column_to_rownames(var="id")
data_raw <- data_raw[,-1]

# convert to inbreedR
library(inbreedR)
data_inbreedR <- convert_raw(data_raw)

# calculate sMLH
smlh <- sMLH(data_inbreedR) #sMLH
```

Sweet, we got some sMLH values now!

Let's do some exploration with it. Answer the following:

1. What are the highest and lowest sMLH values?
2. What is the mean, median and mode sMLH?
3. What does it mean to have a negative sMLH (sMLH < 0)?
4. Plot the distribution of sMLH in a figure (histogram)
5. Why is the distribution not smooth? We see gaps between the bars of the histogram, why?

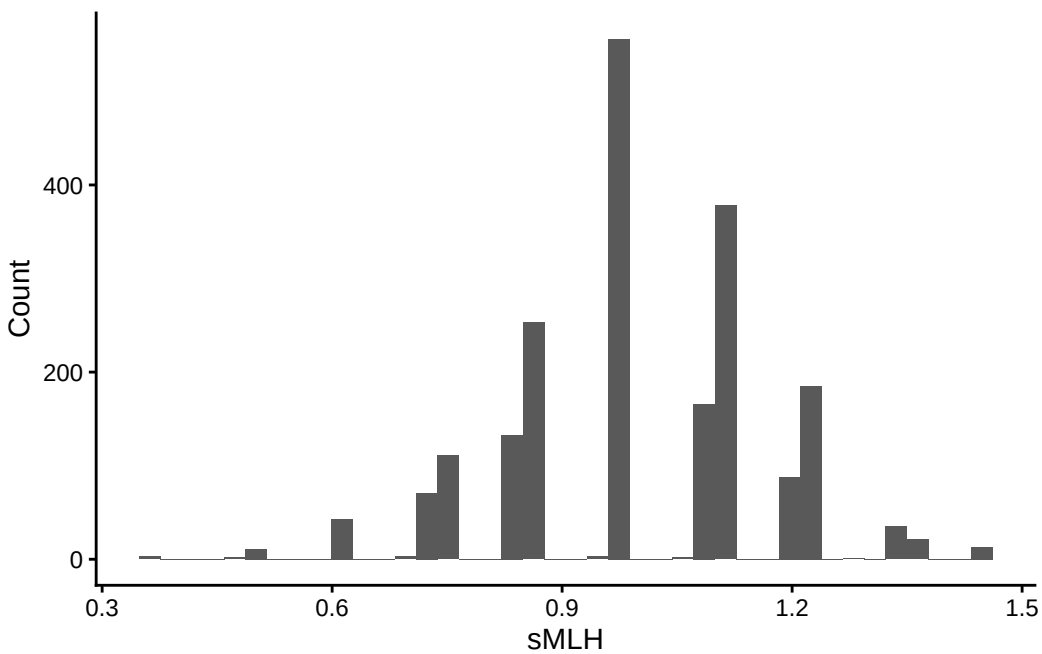
Code and answer

```
## 1. highest and lowest
summary(smlh)

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.3618  0.8632  0.9865  1.0001  1.1099  1.4472

# or:
min(smlh)

[1] 0.3617918

max (smlh)

[1] 1.447167

## 2 averages
mean(smlh)

[1] 1.000059

median(smlh)

[1] 0.9865481

# no built in function for mode:
# install.packages("DescTools") -> only install if not already installed
library("DescTools")

Warning: package 'DescTools' was built under R version 4.3.3

Mode(smlh)

[1] 1.109867
attr(,"freq")
[1] 322
```

```r
# or use a custom function
getmode <- function(v) {
   uniqv <- unique(v)
   uniqv[which.max(tabulate(match(v, uniqv)))]
}
getmode(smlh)
```

```
[1] 1.109867
```

```r
# 4. figure
library(ggplot2)
ggplot(as.data.frame(smlh), aes(x = smlh)) + geom_histogram(bins=40) +
 ↪  theme_classic() +
  labs(x = "sMLH", y = "Count")
```



1. min = 0.3618, max = 1.4472
2. mean = 1.00, median = 0.99, mode = 1.11
3. A negative sMLH means the individual has a lower heterozygosity compared to the population average (i.e. more inbred)
4. sMLH is a continuous variable, but we don't get a smooth distribution. Actually, when we check out the data more closely using e.g. `summary(as.factor(smlh))` (this forces the sMLH values to be categorical, and we see how many values are in each category), we see that there are specific sMLH values that individuals have.

This is an 'artifact' as to how sMLH is calculated. sMLH is a proportion of the number of microsatellites that are heterozygous (because we divide by the number of successfully genotyped loci), and therefore there are only a set number of sMLH values that an individual can have. The more loci, the higher the resolution, and the more possible sMLH values a population has. For example, if there is just one locus, sMLH values can be 0 or 1. If there are two loci, sMLH values can be 0, 0.5 and 1, and so on.

4.7 Exercise 3 (day 2)

Now that we have these values, we can do some analyses and plotting, and ask some research questions. We have the following data: sMLH values from 2078 individuals. These individuals were sampled at 12 different locations / from 12 populations, of which some allow recreational hunting (see the pops file). We have the exact coordinates of each population as well (see the pops file).

The individuals belong to three 'categories': males, females and chicks. Male IDs start with a D. Female IDs start with an H and do not have an appending number. Chicks also start with an H but the way they are named is their mother's ID and then an appending number (e.g. `H462.4`) separated with a dot (.). This means that this chick was the fourth chick of mother H462. Some chicks had unknown numbers and have an ID something like `UNK9`. For some chicks, a year is also appended: this happened when females had clutches across multiple years (e.g. `H441.7_2005`). At the bottom of the dataset there are also some chick IDs with unknown data and are instead named after the location they were found (e.g. `saaRTON.1`)

You will find an additional dataset with metadata from the population, it is stored in `data/week_5_inbreeding/metadata_individuals.csv`. Here you will find the sex of the individual, but also the year of the sample, the density of that population in that year. There are more individuals in this metadata because all chicks of all broods are included: in the previous microsatellite dataset, only one chick was included per brood as they are related.

Your next task: phrase a research question and answer it!

You will first have to phrase a research question which you find interesting: e.g. relate inbreeding to one of the other variables. You'll need to reshape the data to put it in a format that you can analyse (e.g. merge some data, or subset). To answer your research question, you can use some basic comparisons, for example with a t-test or correlation test. To wrap it up, make a graph/figure that visualises the result of your test. And remember to annotate your code well to document what you're doing. You have full flexibility, and we are here for assistance with your question and advise on how to code it, and also how to interpret the data.

Remember you have the entire internet at your fingertips: use google, and you can use ChatGPT (or similar) for assistance as long as you understand what the code is doing! You can ask

it for analytical advice, to help you interpret the output. To maximise your understanding, you can explicitly prompt it to guide you through the process. If you want a real human to double-check what ChatGPT is saying, feel free to ask our help. Allowing the use of ChatGPT is not an approach of lazy teaching, but rather to teach you to use it to your advantage while still staying critical and to make sure you stay smarter than ChatGPT (i.e. don't just copy paste without understanding the code).

Note: this order of execution - calculating something and only afterwards phrase a research question - is not a good research approach, we're just doing this for the sake of the practical ;)

! To finish the practical, present your mini-analysis to me (informally). Make sure to have the following:

1. A research question and hypothesis
2. A small analysis to answer this question (e.g. a t-test, correlation)
3. A figure to show the results (using e.g. `ggplot()`)
4. The code in an organised way, from the start of importing data to the end which shows the figure. Label and annotate your steps!

Here are some tips to make your code cleaner: - Use the hashtag `#` to make a comment (this text won't be executed). - You can use 4 hashtags before and after text to mark a header like this `#### Header ####`. You can quickly get to a header within RStudio (bottom of the script). If you add another hashtag at the end, it will insert a sub-header.

5 Week 9 NGS

6 Day 1: sequencing methods, fasta files and reference genomes

6.1 Overview

In today's lecture, we learned about different sequencing technologies and the different bioinformatic steps that are taken in a typical workflow in molecol/popgen.

The way that data are normally delivered are in fastq format. We'll do some exploration with this data format so you get a feel for these type of files. In this practical, we will be working less with code because at this bioinformatic stage, R is generally not used. Instead, we use the command line which uses UNIX/bash. However, teaching you a second programming language in a single course is a bit of an overkill. We will therefore keep this practical mostly descriptive, test how much knowledge you retained from the lecture, and we will help you understand the files we're working with and how information is stored.

6.2 Exercise 1: sequencing

We started the day learning about different sequencing techniques. These days, it's hard to imagine that Sanger sequencing was used to sequence the entire human genome! This sequencing technique was a breakthrough and is known for its high accuracy, but it's not a surprise that it took more than a decade to assemble an entire genome (rather than just a single gene or genetic variant).

There is a very extensive bioinformatics handbook online that describes DNA sequencing quite well. Read through the part about Sanger sequencing and answer the following questions:

- Explain in your own words how a gel imagine like the one in 16.5 can be used to identify a DNA sequence

Answer

In Sanger sequencing, DNA polymerase synthesizes a new DNA strand using normal dNTPs, but occasionally incorporates a dideoxynucleotide (ddNTP).
ddNTPs lack a 3 hydroxyl group, which means that once incorporated, no further nu-

cleotides can be added — the chain is terminated.
Each of the four ddNTPs (ddATP, ddCTP, ddGTP, ddTTP), on the x-axis here, is labelled with a different fluorescent dye, allowing us to identify which base caused termination. Side note: this separation of bases was common using an old method where you would have four lanes, one per base. Now, all four bases are run in one capillary and we use fluorescent dye / colour to identify the base.
The resulting DNA fragments differ in length depending on where termination occurred. During capillary electrophoresis, shorter fragments migrate further than longer fragments. By reading fragments from shortest to longest and identifying the fluorescent label at each position, we reconstruct the DNA sequence one base at a time.
In this case, we see that a fragment ended in A, G, G and C with increasing fragment length, and we can therefore deduce the sequence likely starts with AGGC, and so forth.

- Why is/was it difficult to sequence an entire genome with Sanger sequencing?

Answer

It is expensive and takes a lot of effort to know the sequence of just a short fragment of say these 20 bases. Sanger sequencing typically reads a few hundred basepairs at a time. Imagine doing the whole process for all 1 billion bases! It's laborious, to a large extent manual or semi-automated, and technically challenging. These days, other sequencing methods exist but Sanger sequencing is still used to sequence a specific gene for example.

These days, we mostly use Illumina sequencing or other sequencing by synthesis methods to obtain our genetic data. Continue reading section 16.3 on Illumina Sequencing, and watch the video (the same we watched in the lecture). Look through the question he poses, but note this a level of understanding I don't expect you to have. You can try to answer them to get a better understanding of the process.

6.3 Exercise 2: read data

Fasta and fastq files are the most common way to store (raw) sequencing data.

- What is the difference between a fasta and a fastq file?

Answer

Fasta files can be for nucleotide data (DNA/RNA) and protein data (amino acids) but fastq files are only for nucleotides
Fastq files also contain information about read quality rather than bases alone, and is therefore more standard to use

Sequencing technologies and bioinformatic pipelines often use the inherent complementary binding of nucleotide bases. What would the complementary strand look like for the sequence below?

```
5'--ACTCGACCT--3'
```

Answer

```
3'--TGAGCTGGA--5'
```

and if we were to write it in standard 5' to 3' order:

```
5'--AGGTCGAGT--3'
```

Our friend Eric also describes fastq formats. Read Chapter 17 from his handbook until section 17.2.3 (so finish with answering the question about the ASCII characters). A bit of this is repetition of the lecture - just differently phrased - but we dig a bit deeper into the headers and how we measure base quality.

Answer the two questions that Eric asks: - Find the characters corresponding to base quality scores of 0, 10, 20, 30, and 40. Remember that the base quality score is the character's decimal representation minus 33.

- Why do you think the scale starts with ASCII character 33?

Answer

- The characters are !, +, 5, ? and I for quality scores 0, 10, 20, 30 and 40, respectively
- The point of this system is to have one symbol per base, therefore the first 'one symbol' character starts at 33.

6.4 Exercise 3: explore fq.gz file

Let's get our hands dirty! As mentioned, fasta files are generally handed outside of the R environment. But that doesn't mean we can't have a look at some of the data in R to get a better feeling of it.

The dataset described by Eric was imported, and we will follow his tutorial with slight modifications here. I only copied the first thousand lines of the fq.gz files. You don't have to look at the tutorial from Eric at the same time, all you need is in this one.

- Load in the chinook data, one dataframe per file (you can call it e.g. R1 for read 1 and R2 for read 2). They can be found in the `data` folder under `week_9_NGS` with names `Battle_Creek_09_chinook_R1.fq.gz` and similar for the reverse strand. First: use the `read_lines` command from the tidyverse package without specifying anything else. Check out the data. That's a bit of an annoying format, right? What happened?

Code and answer

```
library(tidyverse)
R1 <- read_lines("../data/week_9_NGS/Battle_Creek_09_chinook_R1.fq.gz")
R2 <- read_lines("../data/week_9_NGS/Battle_Creek_09_chinook_R2.fq.gz")
```

That is indeed annoying to work with. Unlike excel tables where we have different variables in different columns, fastq files don't look like that. Data are stored sequentially, row-by-row. Every one in every 4 rows represents the header, then we get the sequence, the `#`, and then the read quality. Inconvenient!

What we can do is use the `matrix()` function to force the data to get stored into different columns. That way, we can force lines 1-4 into 4 columns, and then move onto the next line. What we end up with is a matrix where each row represents a read, and the columns the headers, sequence, hashtag symbol, and read quality. Just use the 'annoying' format from above and 'pipe' it using `%>%` with the following code: `matrix(ncol = 4, byrow = TRUE)` and store it into a variable.

Code and answer

```
R1 <- R1 %>% matrix(ncol=4,byrow=T)
R2 <- R2 %>% matrix(ncol=4,byrow=T)
```

Obviously, the column that just says `#` is a bit useless. Remove it in the next step, and force it to be a dataframe instead of a matrix so we can access variables easier. Also give the columns headers.

Code and answer

```
R1 <- R1[,-3] %>% as.data.frame()
names(R1) <- c("header", "sequence", "quality")
R2 <- R2[,-3] %>% as.data.frame()
names(R2) <- c("header", "sequence", "quality")
```

Check out the header of the first sequence. That's a lot of information! Different sequencing companies and sequencing technologies will give you different types of information. What we're looking at here is the following:

`@E00430:101:HKT7WCCXY:1:1101:6411:1204 1:N:0:NGATGT`

We have the first section of the header, with information separated by a colon `:`, and then a second section separated by a space. That's complex!

This is what the different elements mean: - E00430 = the machine - 101 = the run - HKT7WCCXY = the flowcell - 1 = the lane - 1101 = the tile - 6411 = the x - 1204 = the y - 1 = which read of the pair - N = has the data been filtered (Y/N) - 0 = control number (0 on HiSeq and NextSeq machines) - NGATGT = barcode on the read

It would be good if we can access all these different elements separately, so they are stored in different columns. Use the package `tidyr` and its function `separate` to break the string of the header into columns. You can use `?separate()` to find the instructions. Only if you can't figure it out with the instructions, google around to help you.

Make sure to remove the original headers, and to make sure the data types are correct (i.e. numers are indeed numbers, not strings)

Tip

- To use the `tidyr::separate()` function using the name of the column you want to separate in the first argument, followed by using the arguments `into` and `sep` to define the names of the columns it should create and what character should define how to split the string of the column you want to separate
- Use the `remove` argument to remove the original column
- Use the `convert` argument for the function to make the best guess what type of data things are (instead of using the default string)

Code and answer

```
R1 <- R1 %>% separate(header, into = c("id", "part2"),sep = " ") %>%
  separate(id,
           into = c("machine", "run", "flow_cell", "lane", "tile", "x",
            ↪  "y"),
           sep = ":",
           remove = TRUE, convert = TRUE)

R1 <- R1 %>% separate(part2,
                      into = c("read", "filter", "cnum", "barcode"),
                      sep = ":",
                      remove = TRUE, convert = TRUE)

# same for R2
```

```
R2 <- R2 %>% separate(header, into = c("id", "part2"), sep = " ") %>%
    separate(id,
             into = c("machine", "run", "flow_cell", "lane", "tile", "x",
              ↪  "y"),
             sep = ":",
             remove = TRUE, convert = TRUE)

R2 <- R2 %>% separate(part2,
                      into = c("read", "filter", "cnum", "barcode"),
                      sep = ":",
                      remove = TRUE, convert = TRUE)
```

We learned how to interpret the Phred-scaled quality scores with this ASCII-character chart. It would be ideal if the symbols in the `quality` column would actually show us the values, right?
Luckily, these ASCII-characters are standard and common, and there's a built-in R function for this! Use `utf8ToInt()` to get the associated numeric values of the symbols: !, +, 5, ? and I. ::: {.callout-tip title="Code and answer" collapse="true"}

```
utf8ToInt("!+5?I")
```

Woah, that was easy! But… these don't translate to the Phred-scaled values yet right? Make the necessary adjustment in the code to translate into their Phred-scaled scores.

Code and answer

```
utf8ToInt("!+5?I") - 33
```

So, each row represents a read, with a specific sequence and for each base called, a Phred quality score. Do/answer the following:

- How long are the sequence reads? I.e. how many bases per sequence?
- Create a column that calculates the mean Phred score per read. This is quite advanced coding, so here is what the core code looks like `mutate(mean_qual = map_dbl(.x = quality, .f = function(x) mean(utf8ToInt(x) - 33))`. Now you just have to use this code to create a new column
- Do the same to calculate the minimum and maximum Phred score for each read - to they vary among reads?
- Make a plot with the distribution of mean quality scores (just of the forward or reverse strength). Describe the distribution in a 1-2 lines.

Code and answer

```
# how long are the reads?
nchar(R1$sequence[1])

# calculate mean phread + min + max
R1 <- R1 %>% mutate(mean_qual = map_dbl(.x = quality,
                                        .f = function(x)
                                         ↪  mean(utf8ToInt(x) - 33)),
                    min_qual = map_dbl(.x = quality,
                                        .f = function(x) min(utf8ToInt(x)
                                         ↪  - 33)),
                    max_qual = map_dbl(.x = quality,
                                        .f = function(x) max(utf8ToInt(x)
                                         ↪  - 33)))


R2 <- R2 %>% mutate(mean_qual = map_dbl(.x = quality,
                                        .f = function(x)
                                         ↪  mean(utf8ToInt(x) - 33)),
                    min_qual = map_dbl(.x = quality,
                                        .f = function(x) min(utf8ToInt(x)
                                         ↪  - 33)),
                    max_qual = map_dbl(.x = quality,
                                        .f = function(x) max(utf8ToInt(x)
                                         ↪  - 33)))


ggplot(R1, aes(x = mean_qual)) + geom_histogram(bins=20) +
 ↪  theme_classic()
```

These reads are 151 bp long (standard is 150). The minimum and maximum Phred scores for all reads are 2 and 41, respectively The majority of reads have quality scores above 35, with a few poor quality reads with quality scores around 25, equivalent to a probability of error of 0.003 / 1 in 333 error.

After exploring raw reads, we can remove reads that have a low quality. This is one of many sequence filtering steps you can do before aligning to a reference genome. What would you do - would you remove any of the reads in our dataset?

Answer

There's no right or wrong here!
There are many steps that researchers can make when processing their data. Some things are standard, but most things depend on what you want to do with the data. In this case, I think a mean quality score of 25 is still pretty good. We wouldn't want to waste precious data!
But we could, for example, see how many bases have a poor quality. Are there multiple, one after another, or just one with a very low quality?
You wouldn't explore the data in this much detail when you're looking at millions to billions of reads though. Most of the time, the software used in the next steps can be used to filter for e.g. read quality, or incorporates the quality information in its analysis.

6.5 Exercise 4: reference genomes

Even though we do not strictly need a reference genome to align sequences, it makes the process a lot more reliable and easy. Plus, a reference genome enables us to know more about the genes if it is annotated. For example, without a reference genome, we wouldn't know where base 1236123 on chromosome 2 is located - is it in a gene? If so, what kind of gene is it? This is of course very valuable information.

There is a large public database that stores genetic data, including reference genomes. This database is called NCBI. It's a large website, with lots of information.

At the start of this course, you chose a species to design a mini-project for. Do the following:

- Check whether your chosen species has a reference genome on NCBI. A reference genome is also called an assembly. If there is no assembly for your species, find another species that does have a reference genome to answer the rest of the questions.
- When was it created? How large is the reference genome of your species (i.e. how many bases)? Is there just one (or none) reference genome, or multiple?
- Browse around the page. How many chromosomes does your species have? How many base pairs is the largest chromosome (or the largest scaffold)?
- If you were to design a project with this species, do you think the genome assembly is good enough?

Tips

- In the search bar, type the common or latin name of your species and filter for 'Assembly'
- In the URL, you will see that (if your species has an assembly), the URL will be

something like `ncbi.nlm.nih.gov/datasets/genome/GCA****`. The `GCA` is something typical of genome assemblies, it's the accession prefix (allegedly stands for GenBank Assembly, but not sure where the C is coming from)
- You will find lots of metadata on the page of the assembly: genome size, how many pieces it consists of (chromosomes or scaffolds), GC percent, coverage of the genome, just to name a few

6.6 Exercise 5: alignments

When we align fq files to a reference genome, we get a .sam file. Generally, we don't do a whole lot with these files as they just serve as an intermediate step before we detect genetic variants. So - no need to understand this in fine detail.

Depending on the time, read through chapter 17.4 from the bioinformatic handbook from Eric. Don't worry about executing the coding examples.

Answer the following questions: - What is the difference between a .sam and a .bam file? - What column in a SAM file tells us something about the reference sequence that the read aligned to?

Tips

- A .bam file is the binary version of a .sam file
- The @SQ header: "typically the most abundant SAM header lines, each of these gives information about a sequence in the reference genome aligned to"

That's it for today! In tomorrow's lecture and practical, we will focus on identifying genetic variants.

:::

7 Day 2: single nucleotide polymorphisms

In today's practical, we will explore SNPs and work with the files in which genetic variants are normally stored in (a .vcf file). Similar to yesterday's practical, we don't typically work in R when dealing with SAM, BAM, and .vcf files - this is generally done in UNIX/bash but we will try to explore them in R for the sake of the practical. The files that we are typically working with are huge, which would be slow to work with in R. Nevertheless, it's not impossible and we can handle part of the files within R.

7.1 Overview

7.2 Exercise 1: loading in .vcf files

There is a .vcf file containing SNPs located on chromosome 30 stored in the `data/week5_inbreeding` folder, called `snps_chr30.vcf.gz`. The `.gz` means it is gzipped to reduce its size.

Load this vcf file into R. Use both the fread() from the `data.table` package and the read.table() commands and compare the dataframes. What is the difference?

Code and answer

```
vcf_1 <- data.table::fread("../data/week_5_inbreeding/snps_chr30.vcf.gz")
vcf_2 <- read.table("../data/week_5_inbreeding/snps_chr30.vcf.gz")
```

Using `fread()`, we get a dataframe with 21,979 rows and 2 variables, using `read.table()` we get 3786 rows and 245 variables. The difference is that fread included the metadata (starting with ##) and the header (starting with #) which resulted in a weird way of storing the data, and we can actually only see the headers. This is because it 'stopped early' because it expected 2 fields. We could change this by setting fill=TRUE within the command, but it still reads the headers and stores the data in an inconvenient way. If we use `read.table`, the metadata gets excluded automatically and we only see the actual data.

Even though the metadata gets stored in a way that we cannot work with it, it still contains valuable information. Answer the following:

- Are we working with genotypes or genotype likelihoods in this file?

- Can we figure out what species we are working with from the metadata?
- In the metadata, we can see information about the INFO field etc., followed by a long list of rows that start with `contig`. What information do you think we can find here? Tip: look up what a contig, scaffold and chromosome is.

Answers

- In the 5th row, it states "##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">" This means that the format of the data in the 'GT' slot is a genotype, not a genotype likelihood. If we were working with likelihoods, we would instead see something like"GL" and "Description = Genotype likelihood". Alternatively, you could check out the actual information about the loci, which is stored in binary format which indicates it is a genotype.

- Typically, this info would be stored in the first few rows. Unfortunately, that information is missing here. But, if we browse through the metadata, we also see a lot of other information is stored. Including the actual command used to call the variants using the tool bcftools. This line can be found in the 30th row of the metadata.

```
bcftoolsCommand=mpileup -f /vol/cluster-data/rchen/annotation/PO2979_Lyrurus_tetrix_black_g
-a DP -C 50 -q 20 -b /vol/cluster-data/rchen/wgr/chicks_genomes/output/list_bam_all.txt
-I --threads 12
```

Even though we will not go into the details of how this is typically done or what any of this means, it is good to know that we can find a lot of useful information in this metadata header. So this was a bit of a trick(y) question: we can see from this command that a certain Latin name is mentioned used for variant calling. Which is *Lyrurus tetrix*, followed by its common name the black grouse. Which is the data we're working with, what a surprise!

- A contig is a stretch of a contiguous DNA fragment, meaning there are no gaps between the bases. A reference genome typically consists of information on each chromosome. When contigs are combined, they form a scaffold. So: contigs are contiguous, gapless DNA sequences; scaffolds link multiple contigs together (often with unknown gaps 'N'); and chromosomes are the highest level, representing whole, ordered DNA molecules. The information we can find here (e.g. the 34th row `##contig=<ID=ScEsiA3_1;HRSCAF=11,length=5887>`) starts with the name of the contig `ScEsiA3_1;HRSCAF=11` which forms part of scaffold 11, and its length measured in basepairs. This is information that comes from the reference genome. The name of the contig here is also used in the rest of the .vcf file when we are looking at specific basepairs on each contig (i.e. each row within the actual data part of the file)

7.3 Exercise 2: cleaning the file

Although the metadata is useful, we will stop looking at it now and instead focus on the genotype data. I.e. you can get rid of the `vcf_1` dataframe and only look at `vcf_2`, perhaps we can also rename it.

Next, we want to add some column names to make it easier to deal with the genotype data. Rename the first 9 columns (the columns that do not contain genotype data) in the dataframe. In a random order, this is the data that can typically be found in a .vcf file (recap from the lecture): info about the location of the SNP, the reference and alternative allele, quality score, whether data were filtered, the info column, the format in which the genotypes are presented in. Look up the usual order and names. Tip: there are two columns that only contain NA's (represented as dots .): remove them.

Code and answer

```
# remove vcf_1
rm(vcf_1)
# rename vcf_2 to vcf
vcf <- vcf_2
# remove vcf_2
rm(vcf_2)

vcf <- vcf[,-c(3,7)] #remove columns 3 and 7 which are empty / all NA
names(vcf)[1:7] <- c("CHROM", "POS", "REF", "ALT", "QUAL", "INFO",
 ↪  "FORMAT")
```

7.4 Exercise 3: handling and filtering

1. How many SNPs are in this vcf file?
2. And how many individuals are in the file?
3. On chromosome/contig `ScEsiA3_18236;HRSCAF=21586` at position `14454`, what basepair does 1 refer to and what genotype does an individual with coded genotype 0/0 have?
4. If we were to filter this VCF file to make sure we are only working with high-quality SNPs, what could we do?
5. What is a biallelic SNP? Do you think this file only contains biallelic SNPs or also triallelic SNPs?

Code and answer

```
# 1. Each row represents a unique SNP, which means the number of
 ↪  rows gives us the answer as to how many SNPs are in the file
nrow(vcf)

# 2. Each individual genotype is stored in the columns after
 ↪  'FORMAT'. There are 7 columns including 'FORMAT', so we can
 ↪  subtract that from the number of total columns in the file
ncol(vcf) - 7

# 3. Let's first filter the row that corresponds to the mentioned
 ↪  SNP
library(dplyr)
snp <- vcf %>% filter(CHROM == "ScEsiA3_18236;HRSCAF=21586" & POS ==
 ↪  14454)
snp[,1:8] # check out the main columns and the first genotype
# we see that the reference allele at this SNP is C, and the
 ↪  alternative allele is T
# this means that basepair '1' corresponds to T (the alternative
 ↪  allele) and a genotype of 0/0 means that the individual is
 ↪  homozygous for the reference allele, so has a genotype of C/C at
 ↪  this locus/SNP. The individual genotype stored in V10 has a 0/0
 ↪  (C/C) genotype

# 4. We can see in the 'FORMAT' column that information PL and DP
 ↪  are available, these correspond to the normalized phred-scaled
 ↪  likelihood and the read depth, respectively. Common vcf
 ↪  filtering steps are: filtering for quality (PL), minimum and
 ↪  maximum depth (DP), missingness, minor allele frequency,
 ↪  hardy-weinberg equilibrium and linkage. We can see there are
 ↪  missing genotypes (denoted as ./.), so it would be good to
 ↪  investigate this in more detail. Whether we filter for MAF, HWE
 ↪  and LD depends on the analysis we want to conduct. If we want to
 ↪  make sure that very rare alleles are excluded as they might
 ↪  represent a genotyping error, we might want to filter for MAF
 ↪  (e.g. exclude alleles that have a frequency of less than 0.05 in
 ↪  the population).

# 5. A biallelic SNP is a common form of genetic variation at a
 ↪  single nucleotide position in the genome, where only two
 ↪  distinct nucleotides (alleles) are observed across a population.
 ↪  This file only contains biallelic SNPs, otherwise we would have
 ↪  seen multiple variants in the 'ALT' column. If there were
 ↪  triallelic SNPs, we would see two bases in the ALT column,
 ↪  separated by a comma. The genotype column would then include 0,
 ↪  1, and 2's which denote for 0 = reference allele, 1 = 1st
 ↪  alternative allele and 2 = 2nd alternative allele.
```

7.5 Exercise 4: comparing SNPs and microsatellites

Even though SNPs are very commonly used, microsatellite genotyping is still very valuable as well. The method you use depends on the purpose of the study, what you want to find out, and how much budget you have.
The next exercise is not coding based, but literature based.
Find 2 papers published that used both microsatellites and SNPs and answer the following: 1. How many microsatellites were used and how many high-quality SNPs? 2. What filtering steps were performed for the microsatellite and SNP datasets? 3. What was the main research question they addressed? 4. Did they draw different conclusions based on the two different datasets? 5. Do they conclude whether SNPs or microsatellites were more informative or reliable?
Once you have the answers to these questions for both papers, present them to me and you are done for the day!

8 Week 10 ROHs

8.1 Analysis 2 - SNPs

8.1.1 RZooROH package

Now it's time to turn it up a notch: we'll move on from using microsatellite markers to a larger dataset: single nucleotide polymorphisms (SNPs). These genetic variants are most commonly analysed in (population) genetics and molecular ecology.
We will continue using genetic data from the black grouse. We have sequencing data from 190 males (not the same ones as from the microsatellite dataset): the whole genomes of these individuals have been sequenced. This allows us to identify runs of homozygosity: long, continuous stretches of homozygous loci.
This morning, we learned about the RZooROH package we'll be using. To familiarise yourself even more, read chapter 3 and 4 from the RZooRoh vignette. Rest assured: this is a complex package with complex algorithms behind the computation. We do not expect you to understand how this works in fine detail, but we want you to understand broadly what the package is doing.
Importantly, the RZooROH package uses the term HBD: homozygous by descent. For the sake of our analyses, this can be considered analogous to a run of homozygosity (ROH). The difference is that a ROH is what you observe in the genotype data, whereas a HBD segment is the true underlying genealogical state. In other words, a ROH can be homozygous not necessarily because it was inherited through sharing a common ancestor, although it is typically assumed this is the case. In contrast, a HBD segment is a genomic segment where the two alleles truly descend from the same ancestral chromosome copy. All HBD segments are a ROH, but not all ROHs are a HBD segment.
The main things to understand are: - How is the RZooROH probability approach different from rule-based approaches - The best approach depends on the data - HBD segment / ROH segment length tells us more about the demographic history of the population

8.1.2 VCF files

SNPs are generally stored in a `.vcf` format: each row is a locus and there are various columns. The most important ones are: - CHR: this is the chromosome - POS: this is the basepair position within that chromosome - REF: this is the allele that is noted in the reference genome, e.g. T - ALT: this is the variant allele that differs from the reference genome, e.g. C
Then follows a number of columns: one for each individual in the dataset. Each individual

will have a value that looks like this: 0/1:246,0,248:42. Three types of information are stored here, separated by a colon. The information relevant to us now is the genotype which are the first two numbers: 0/1. These two numbers indicate in binary format what two alleles the individual has for this locus/SNP. 0/1 indicates that allele 1 is 0 = reference allele = T and that allele 2 is 1 = alternative allele = C. This individual is therefore heterozygous for this locus. NA's are indicated by dots (e.g. ./.). Homozygous individuals are therefore 0/0 (homozygous reference allele) or 1/1 (homozgyous alternative allele). We will learn more about VCF files later in this course.

8.1.3 Exercise 1

Let's explore the .vcf file before digging into inbreeding estimates. We will only look at one chromosome (chr 30) because we are working with a huge dataset here. You will find this subsetted .vcf file in `data/week_5_inbreeding` named `snps_chr30.vcf.gz`. The `.gz` indicates this file has been zipped to make it smaller in file size.
Execute the following: 1. Import the dataset. The best function to use for this is read.table() from the `data.table` package as it can directly import .gz files. 2. Explore the dataset and try to understand what all columns mean 3. The columns do not have headers. Give the columns the following headers: "CHROM", "POS", "ID", "REF", "ALT", "QUAL", "FILTER", "INFO","FORMAT", followed by the ids stored in the file `vcf_sample_names.vcf`. 4. You can ignore column 8 "info" - we'll get back to this later in the course.

Code and answer

```
# load library
library(data.table)

# import SNP data
snps <- read.table("../data/week_5_inbreeding/snps_chr30.vcf.gz")

# import individual names
names <- read.csv("../data/week_5_inbreeding/vcf_sample_names.txt",
 ↪  header=F)

# change names manually
names(snps) <- c("CHROM", "POS", "ID", "REF", "ALT", "QUAL",
 ↪  "FILTER", "INFO","FORMAT", names$V1)

# explore the data with e.g. View()
```

If you continue read the sections on VCF formats in chapter 5 of the vignette, you'll see

we need to make a conversion of the data (5.2.4) int o a .gen dataformat which is a type of genotype probability format. Read through this, but don't worry - this are steps done outside of R and we have computed them for you. The converted file can be found in `data/week_5_inbreeding` titled `snps_chr30.gen.gz`. We won't go into the details why or how this is done.

8.1.4 Exercise 2

Next, we will follow the steps to define an RZooROH model and identify HBD segments. First, read the vignette on how data should be read / imported (you can skip 5.3.4) until (and including) section 5.3.5.

1. Check out the converted vcf file to know which columns to define. You will need: supcol, poscol and chrcol.
2. Read in the data, keeping in mind we are importing genotype probabilities (gp)
3. Decide whether you want to filter for minor allele frequency (MAF) and implement it accordingly
4. Make sure to specify the sample file
5. How many markers were imported?

Code and answer

```
## load in data
library(RZooRoH)
file_gen <- "../data/week_5_inbreeding/snps_chr30.gen.gz"
data <- zoodata(genofile = file_gen, zformat = "gp", chrcol = 1,
 ↪  poscol = 3, supcol = 5, samplefile=
 ↪  "../data/week_5_inbreeding/vcf_sample_names.txt")
```

3786 markers/SNPs were imported if you don't filter for MAF.

Nice! We now have a zooin object that can be used to estimate HBD segments. You can access different types of information using the `@` symbol (see section 5.3.7).

8.1.5 Exercise 3

Now, we need to define the model, in which we specify the number of segments lengths we want to measure and their lengths. RZooROH and these specifications are complex: complexity that goes beyond the purpose of this practical. We will therefore take the easiest step: we just want to understand which segments are likely to be in HBD and which ones not (section 6.1). We can just read that section and skip to page 24 to see how to define a model with one HBD-class.

- Define the (simplistic) model next.

Code and answer

```
### Define model
model <- zoomodel(predefined=FALSE,K=1,krates=c(10))
```

Code and answer

```
### run model
out <- zoorun(zoomodel = model,
              zooin = data,
              nT = 8, localhbd = TRUE)

### divide out rzooroh ###
hbd <- realized(out)

hbd$file_id <- out@sampleids
```